
UI Architecture Patterns for Apps

Mastering Software Architecture Patterns
and System Design

Rahul Rajat Singh

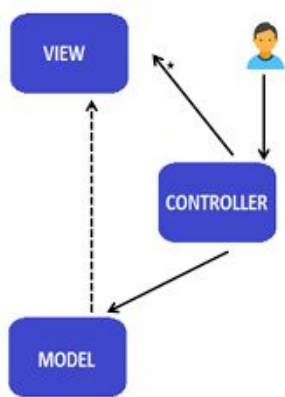
UI Architecture Patterns: MVC, MVP, MVVM

UI Architecture Patterns for Apps

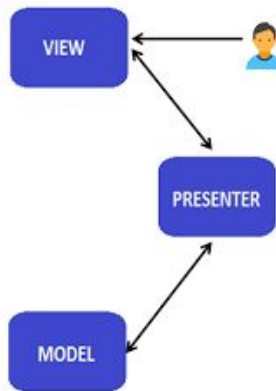
Mastering Software Architecture Patterns and System Design

Introduction to UI Architecture Patterns

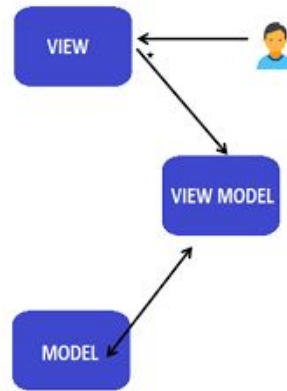
- UI code can become tightly coupled and hard to test
- UI architecture separates concerns and improves maintainability
- MVC, MVP, and MVVM address these problems differently
- These patterns influence how data, logic, and view interact



MVC



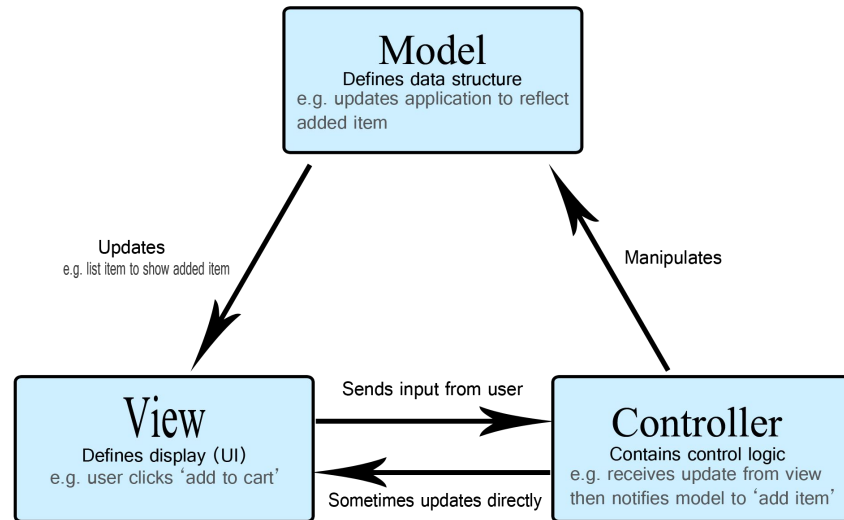
MVP



MVVM

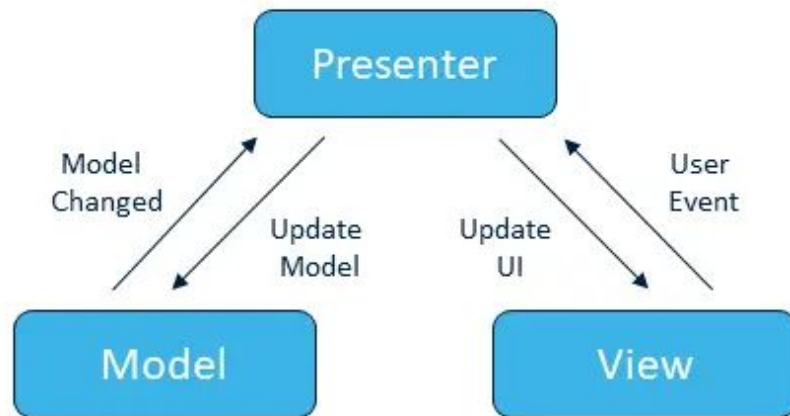
What is MVC (Model–View–Controller)?

- Model = business/data logic
- View = UI layer (what the user sees)
- Controller = handles input, updates model/view
- View & Controller are tightly coupled
- Originated in desktop apps, adopted in web (e.g., Ruby on Rails)



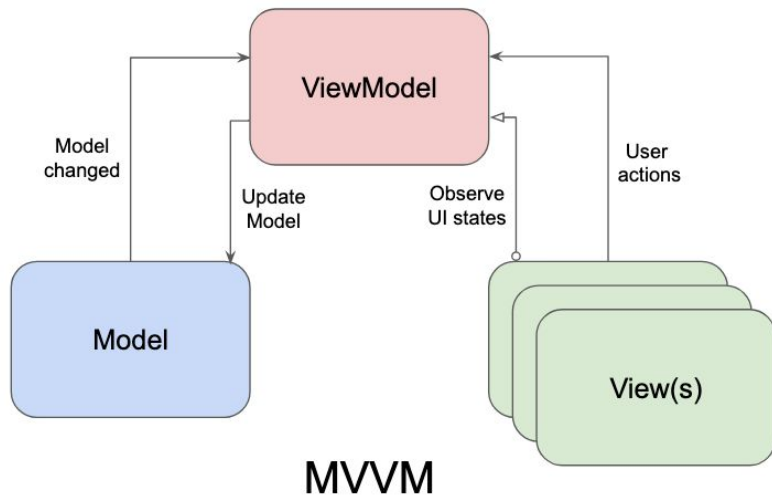
What is MVP (Model–View–Presenter)?

- Presenter handles UI logic & interacts with Model
- View is passive, only responsible for rendering
- Presenter updates View via interface
- Great for unit testing UI logic
- Used heavily in Android development



What is MVVM (Model–View–ViewModel)?

- ViewModel exposes data and commands
- View binds directly via data binding
- No explicit reference from ViewModel to View
- Two-way binding synchronizes state
- Common in WPF, Xamarin, Angular, React (to some extent)

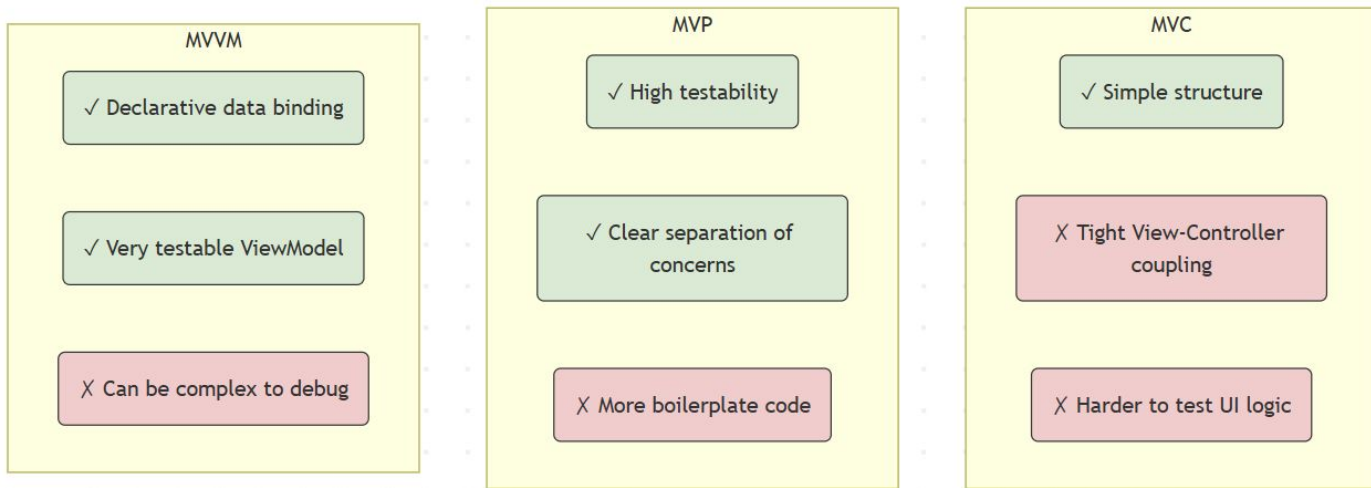


Side-by-Side Pattern Comparison

- MVC (Model–View–Controller)
 - View is active (it handles some logic)
 - Logic is in the Controller
 - Moderate testability
 - Uses manual binding between layers
- MVP (Model–View–Presenter)
 - View is passive (only displays)
 - Logic is in the Presenter
 - High testability due to separation via interfaces
 - Still uses manual communication
- MVVM (Model–View–ViewModel)
 - View is passive, bound to ViewModel
 - Logic resides in ViewModel
 - High testability with minimal glue code
 - Uses declarative (often two-way) data binding

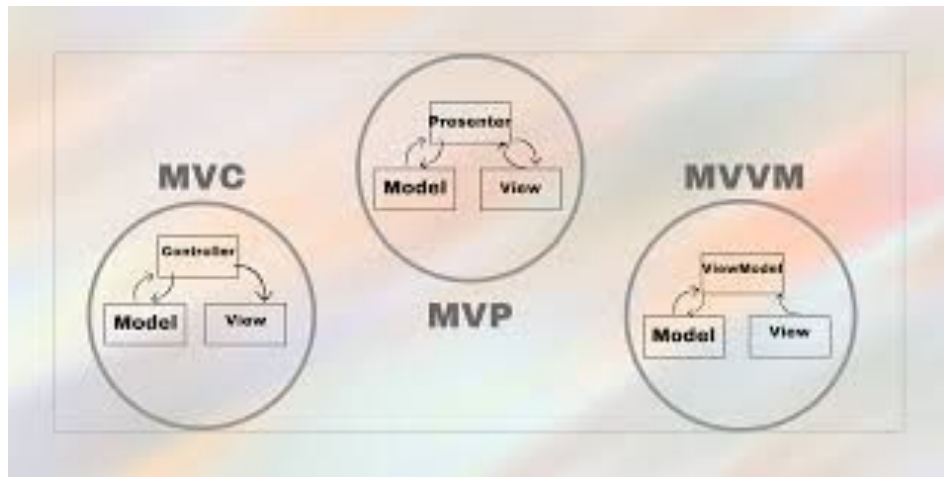
Pros and Cons: MVC vs MVP vs MVVM

- MVC: Simple but often gets messy in large UIs
- MVP: Good testability, clear boundaries, but more boilerplate
- MVVM: Elegant with data binding, but complex for debugging
- Choice depends on framework, team experience, and UI complexity



Pattern Selection in Real Projects

- Web UIs: MVC often dominates (e.g., Rails, Django)
- Android: MVP or MVVM with Jetpack
- React: View + Hooks/State resembles MVVM
- iOS: MVVM common with SwiftUI
- Hybrid UIs: Combine elements of patterns pragmatically



Common Anti-Patterns & Pitfalls

- MVC: Fat Controllers, View-Controller spaghetti
- MVP: Over-abstracted Presenters
- MVVM: Excessive two-way binding → unexpected bugs
- Don't blindly follow patterns — understand why they exist
- Keep UI logic lean and testable

Summary - UI Architecture Patterns: MVC, MVP, MVVM

- Understand why the pattern exists
- Prefer testability, separation, and maintainability
- MVC = simple but limited
- MVP = great for test-driven UI
- MVVM = declarative, modern UIs
- What's next:
 - Unidirectional UIs: MVU & VIPER

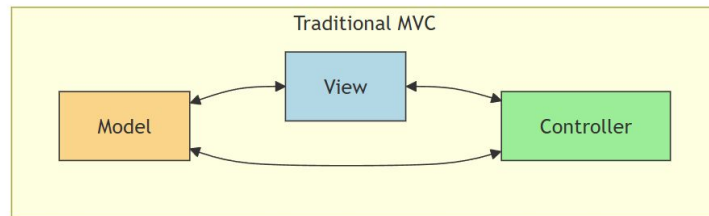
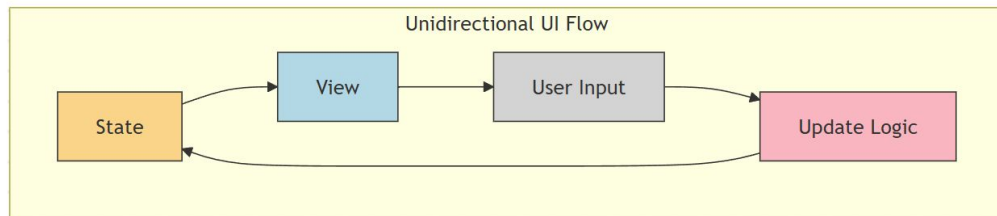
Unidirectional UIs: MVU & VIPER

UI Architecture Patterns for Apps

Mastering Software Architecture Patterns and System Design

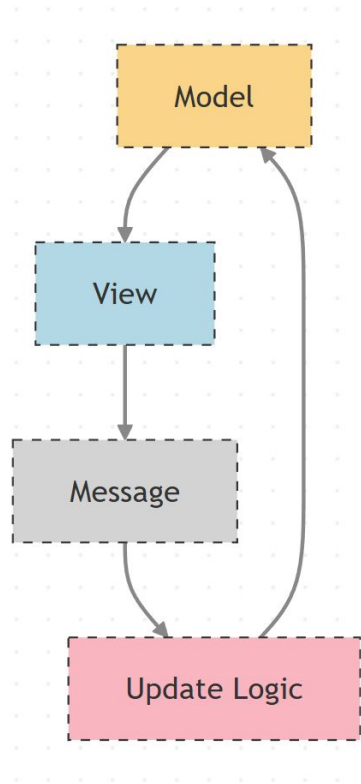
Why Unidirectional UIs?

- Traditional MVC patterns often suffer from tangled flow.
- Modern UIs need predictability and testability.
- Unidirectional flow simplifies state and data handling.
- Popularized by web (Elm) and mobile (VIPER, Redux).
- Today's focus: MVU and VIPER patterns.



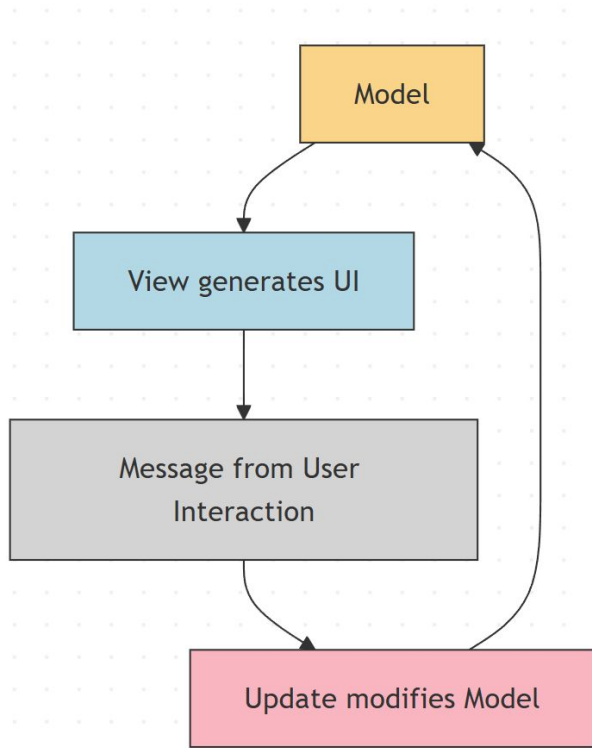
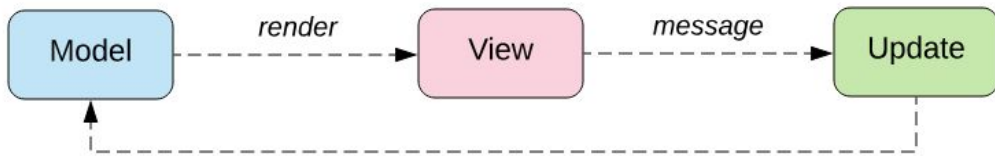
What is Unidirectional Data Flow?

- State is the single source of truth.
- View reflects state — no direct mutation.
- User interactions generate messages (events).
- Messages are processed to update state.
- Cycle ensures predictability and separation of concerns.



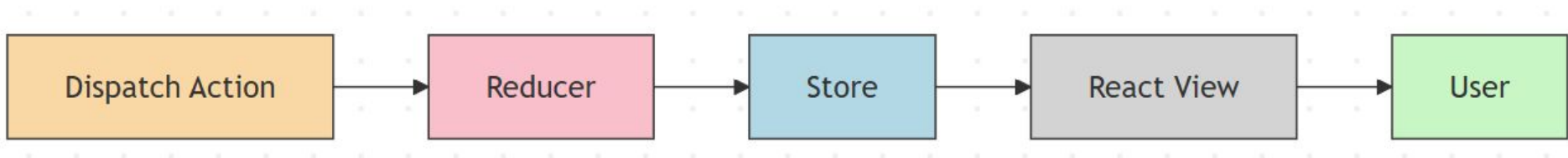
Introducing MVU (Model-View-Update)

- Originated in Elm (a functional web language).
- Core idea: a loop of Model, View, and Update.
- View is a pure function of Model.
- Update takes Message + Model $\rightarrow$ returns new Model.
- Widely adopted via Redux and similar frameworks.



MVU in the Wild: Redux and Beyond

- Redux = MVU applied to JavaScript.
- State managed in a central store.
- Actions = messages, Reducers = updates.
- React View layer complements the model.
- Tools: Redux DevTools, React Hooks, Zustand.

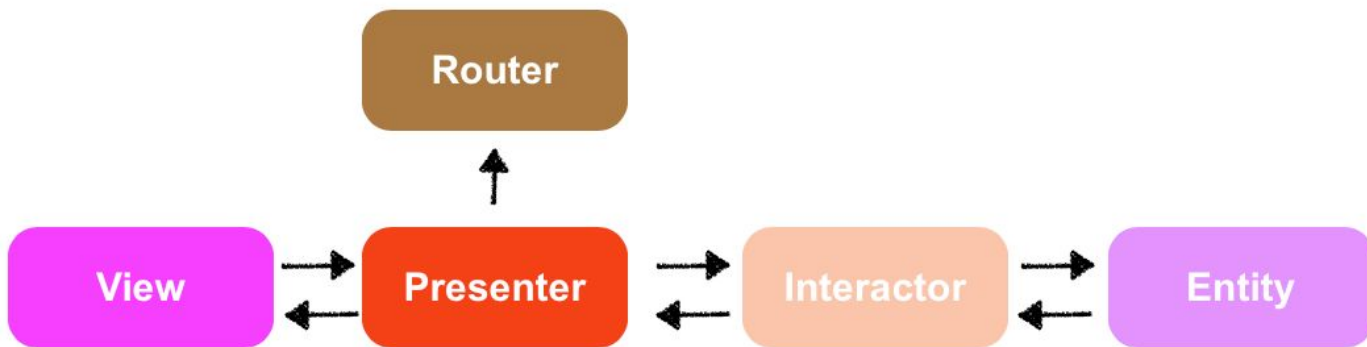


Why MVU Works Well

- Predictable UI behavior.
- Great for testing: pure functions, mockable input/output.
- Time travel debugging.
- Easy to trace bugs and state changes.
- Encourages clean separation of concerns.

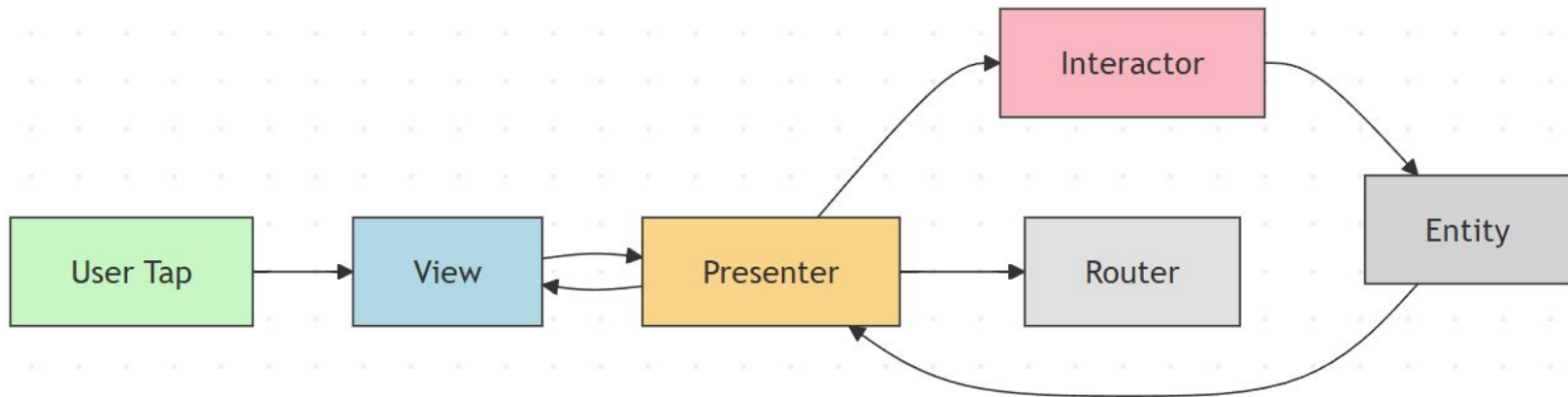
Switching Contexts: What is VIPER?

- VIPER = View, Interactor, Presenter, Entity, Router.
- Clean architecture for iOS (Swift).
- Strong separation of concerns.
- Each component has a narrow responsibility.
- Often used with Clean Swift templates.



VIPER Flow: From Tap to Business Logic

- User taps View → Presenter notified.
- Presenter calls Interactor for business logic.
- Interactor updates Entity or fetches data.
- Presenter formats result for View.
- Router handles navigation, if needed.



Comparing MVU and VIPER

- MVU: Functional, loop-based, best for reactive UIs.
- VIPER: Clean OO pattern, fits mobile app layers well.
- MVU focuses on simplicity, VIPER on modularity.
- Both avoid tangled two-way data binding.
- Choose based on platform, team skill, and testability needs.

Common Pitfalls to Avoid

- Overengineering with VIPER (too many files).
- Improper MVU state modeling → bloated reducers.
- Forgetting separation of view and logic.
- Leaky abstraction: business logic in view.
- Poor tooling or missing structure = chaos.



Too Much VIPER



Bloated Reducer



Business Logic in View

Summary - Unidirectional UIs: MVU & VIPER

- Unidirectional flow improves predictability and testing.
- MVU = loop of model/view/update (great for web/reactive UI).
- VIPER = modular layers (great for mobile/Clean Swift).
- Choose based on team, tech stack, and problem domain.
- Architecture should serve clarity, not complexity.
- What's next:
 - Offline-First & Data Sync

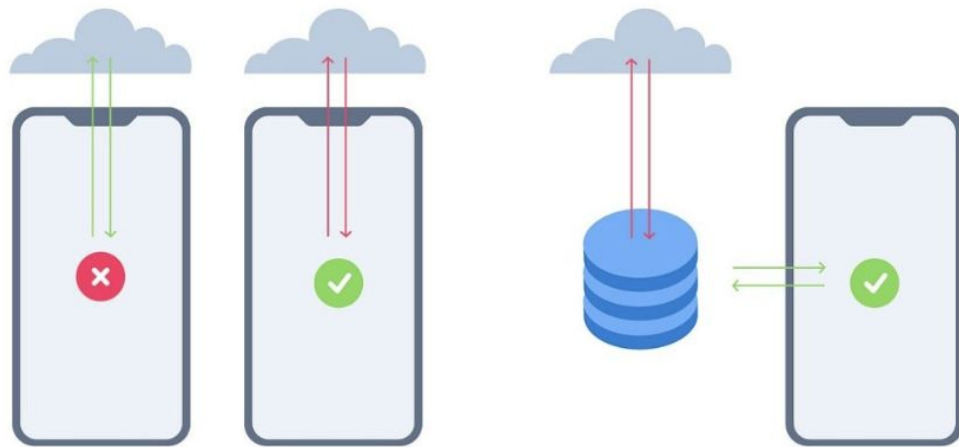
Offline-First & Data Sync

UI Architecture Patterns for Apps

Mastering Software Architecture Patterns and System Design

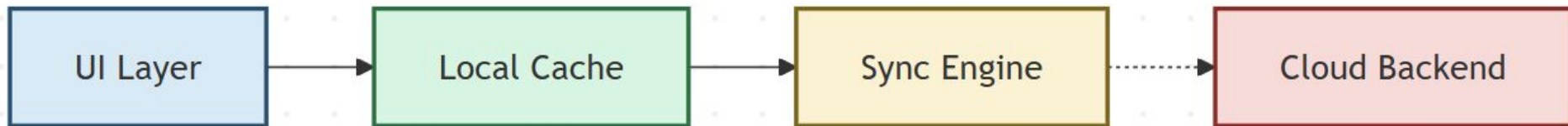
Why Offline-First Matters

- Users expect apps to “just work” even without connectivity.
- Offline-first enhances usability, especially in mobile and remote environments.
- Core challenges: data consistency, sync complexity, conflict management.
- Common use cases: travel apps, field service tools, chat, CRMs.



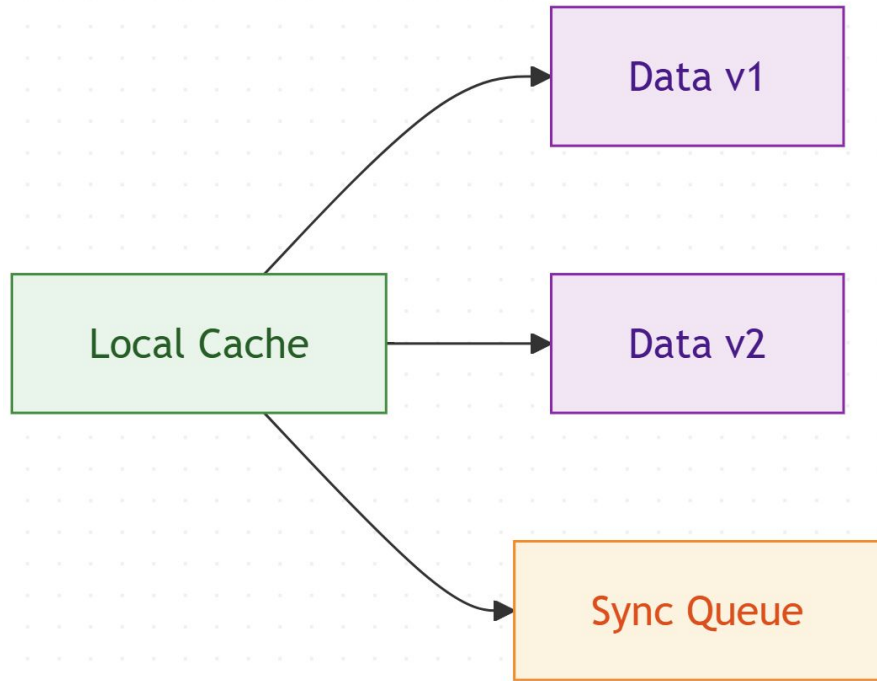
Core Principles of Offline-First Architecture

- Local-first interaction model: reads/writes happen locally first.
- Sync is decoupled and async: background processes handle server updates.
- Data is stored in a local cache (e.g., SQLite, IndexedDB).
- Sync must be resilient to failures and retries.



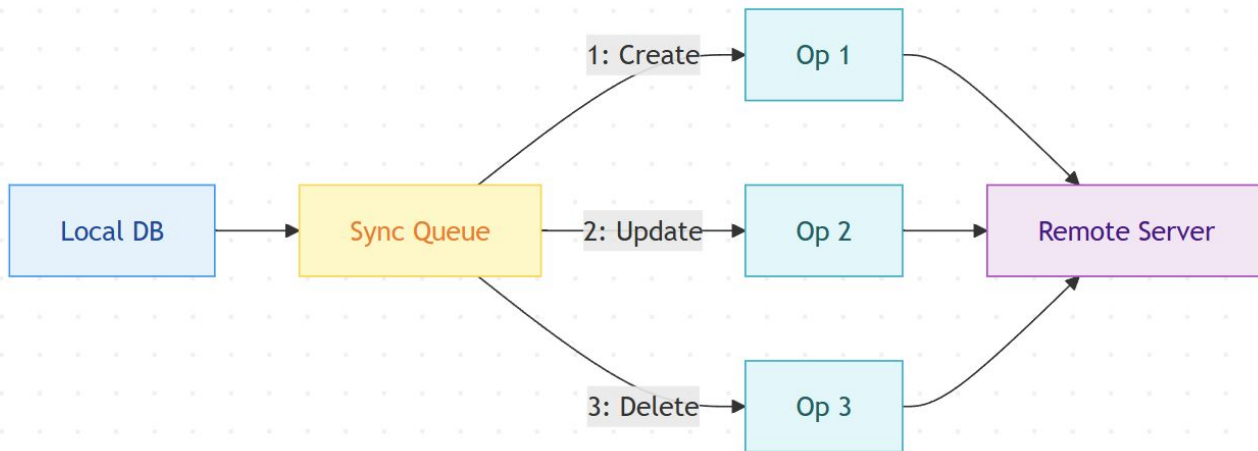
Local Caching: Foundation of Offline Experience

- Enables fast reads/writes without network latency.
- Typically involves structured storage (e.g., Realm, SQLite, Room).
- Requires versioning to track changes.
- Cache invalidation & TTL (time-to-live) can be used for data freshness.



Sync Queues: Handling Outbound Changes

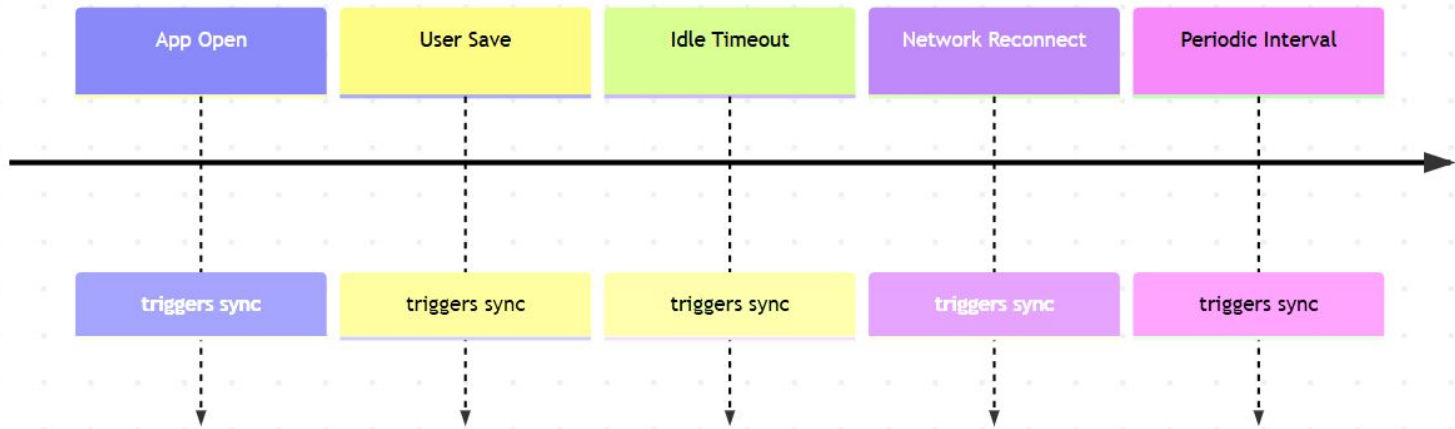
- Local changes are queued for later transmission.
- Each entry contains data + operation (create/update/delete).
- Queue is processed in order, respecting dependencies.
- Retries and exponential backoff handle network failures.



Background Sync Strategies

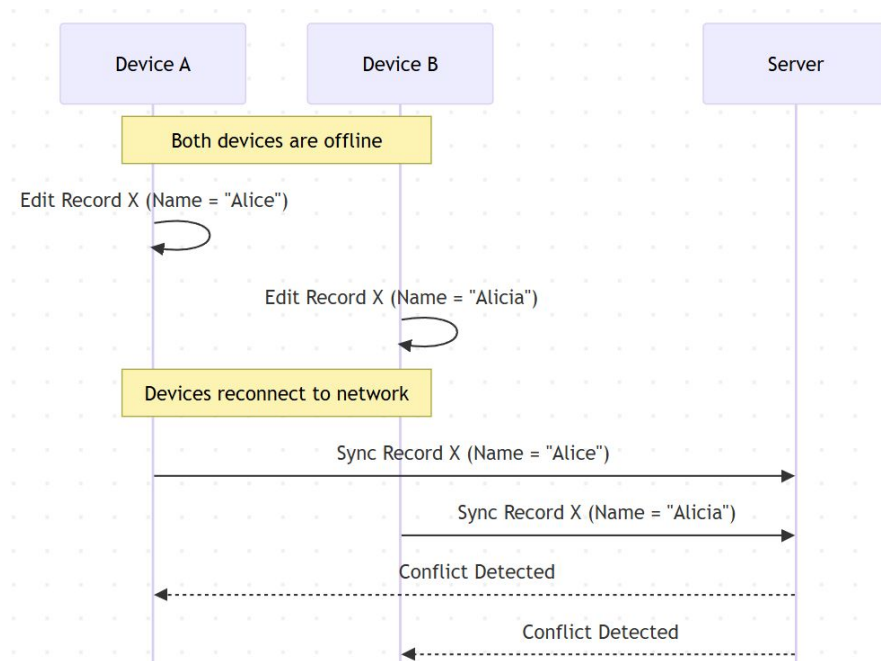
- Periodic sync (e.g., every X minutes or on app start).
- Trigger-based sync (on user action, state change).
- Opportunistic sync (when connectivity is restored).
- Background services (e.g., Android WorkManager, iOS Background Fetch).

Sync Trigger Events



Conflict Scenarios in Data Sync

- Concurrent edits from multiple devices.
- Server changes vs. local changes.
- Network partition and delayed updates.
- Stale data being overwritten.



Conflict Resolution Strategies

- Last Write Wins (LWW): Simple but may cause data loss.
- Merge Strategies: Combine changes if fields are independent.
- Manual Review: Ask user to resolve.
- Operational Transform (OT) / CRDTs: Complex but automatic.

Last Write Wins + Simple -
Overwrites data

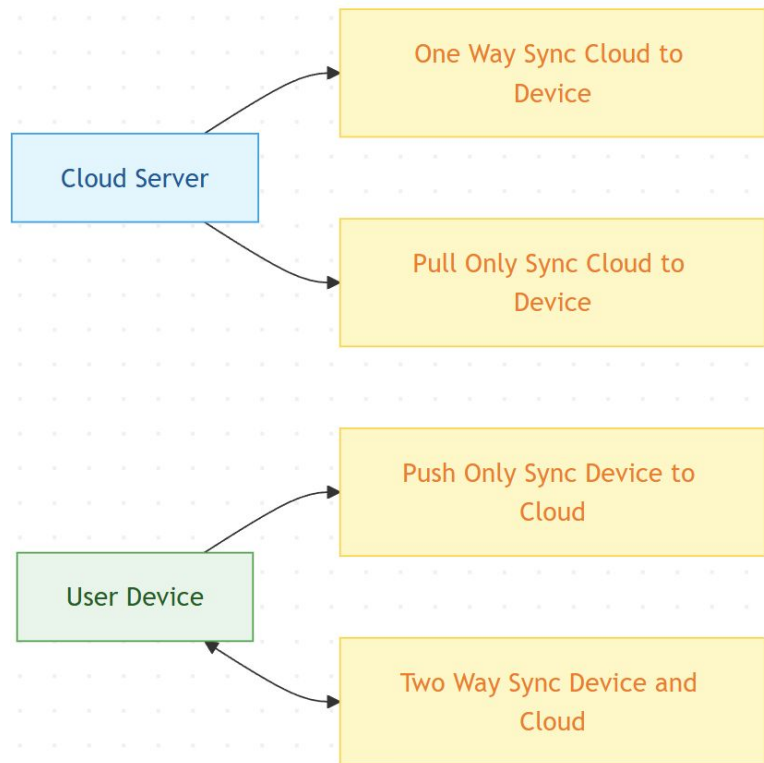
Merge Changes + Keeps all
fields - Complex logic

Manual Review + User
decides - Slower UX

CRDT / OT + Auto conflict
resolution - Hard to
implement

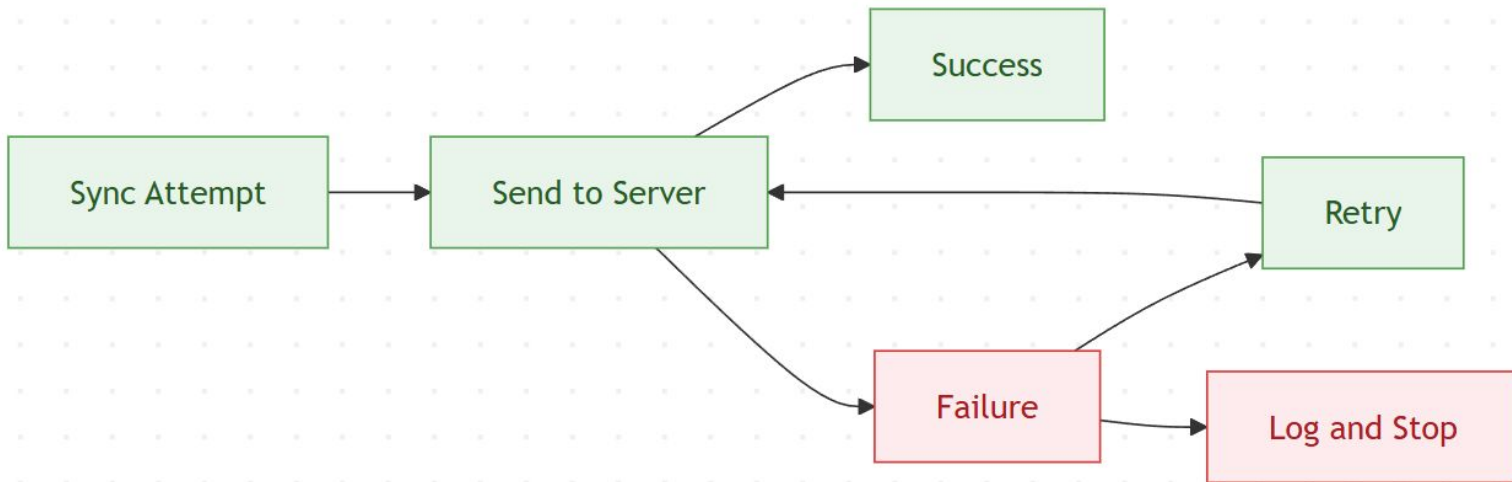
Sync Directionality: One-Way vs. Two-Way

- One-Way Sync: From cloud to device (e.g., news feed).
- Two-Way Sync: Bidirectional updates (e.g., forms, CRM data).
- Full vs. incremental sync.
- Pull-only, push-only, or hybrid models.



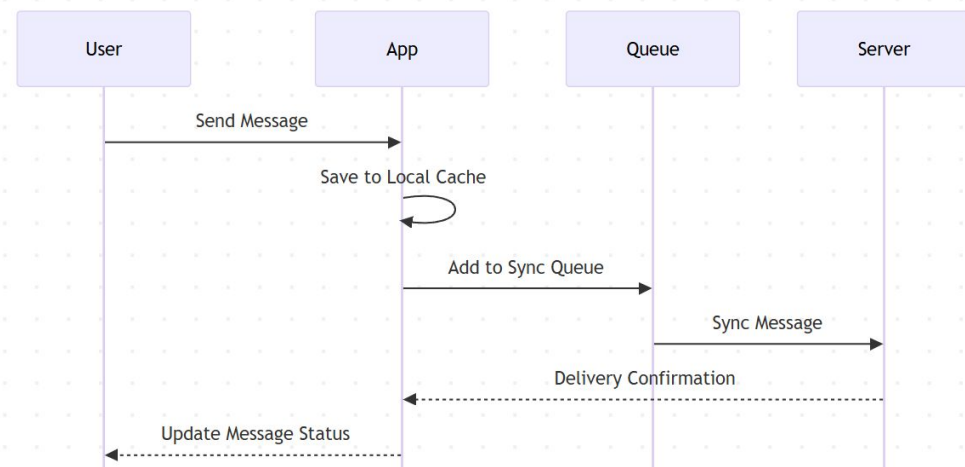
Designing for Resilience & Retry Logic

- Use idempotent operations on server.
- Retry failed syncs with backoff.
- Store failed payloads locally.
- Avoid tight coupling between app logic and sync outcomes.



Case Study: Offline Chat App

- Messages are saved locally first.
- Outbox queue syncs messages when online.
- Delivery status updates via background polling/websockets.
- Handles conflict when two users edit a shared thread offline.



Summary - Offline-First & Data Sync

- Design for offline from the start, not as an afterthought.
- Sync queues, local caches, and conflict resolution are key pillars.
- Prioritize user experience: smooth, predictable behavior.
- Resilient systems = better trust, especially in flaky networks.
- What's next:
 - Selecting UI Patterns by Platform

Selecting UI Patterns by Platform

UI Architecture Patterns for Apps

Mastering Software Architecture Patterns and System Design

Why Platform Matters in UI Architecture

- Different interaction models (click vs. touch vs. keyboard)
- Varying lifecycles & resource constraints
- Native capabilities and performance needs
- Deployment and update cycles differ
- User expectations shift by platform

Web

- Input: Click
- Performance: Moderate
- Deployment: CI/CD

Mobile

- Input: Touch
- Performance: Limited
- Deployment: App Store

Desktop

- Input: Keyboard & Mouse
- Performance: High
- Deployment: Installer

Common UI Patterns – Quick Recap

- MVC: Model–View–Controller
- MVP: Model–View–Presenter
- MVVM: Model–View–ViewModel
- MVU: Model–View–Update (Elm/React-style)
- VIPER: View–Interactor–Presenter–Entity–Router (iOS-heavy)

MVC

- ✓ Separation
- ✓ Testable
- ⚠ Medium Complexity

MVP

- ✓✓ Good Separation
- ✓✓ Testable
- ⚠ Medium Complexity

MVVM

- ✓✓ Strong Separation
- ✓✓ Testable
- ⚠⚠ Higher Complexity

MVU

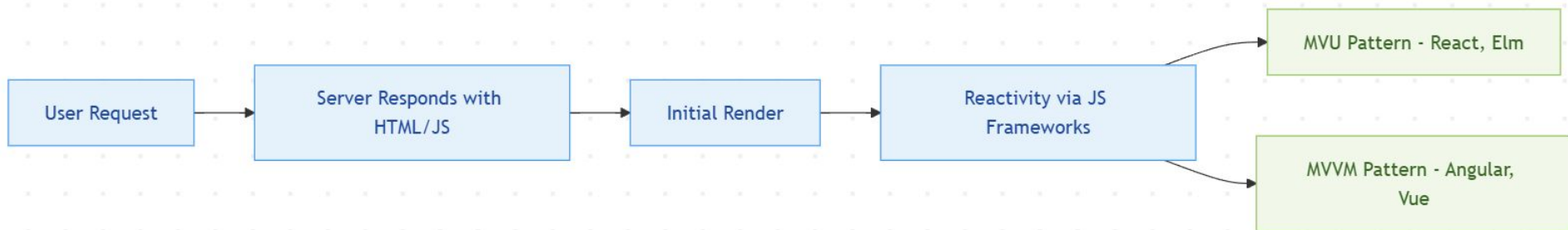
- ✓✓✓ Excellent Separation
- ✓✓ Testable
- ⚠⚠ Higher Complexity

VIPER

- ✓✓✓ Very Strong Separation
- ✓✓✓ Highly Testable
- ⚠⚠⚠ Very Complex

Web UIs – Strengths and Constraints

- Stateless nature favors MVU or MVVM
- DOM as a reactive tree structure (fits declarative UIs)
- Rich ecosystem: React, Vue, Angular
- Frequent updates via CI/CD
- Considerations: SEO, load times, cross-browser quirks



Recommended Web UI Patterns

- MVU (Model–View–Update): Great for React, Elm
- MVVM: Angular & Vue align well
- MVC: Used in older frameworks (Rails, Backbone)
- Modern frameworks often blend these

React Uses MVU

Vue Uses MVVM

Angular Uses MVVM

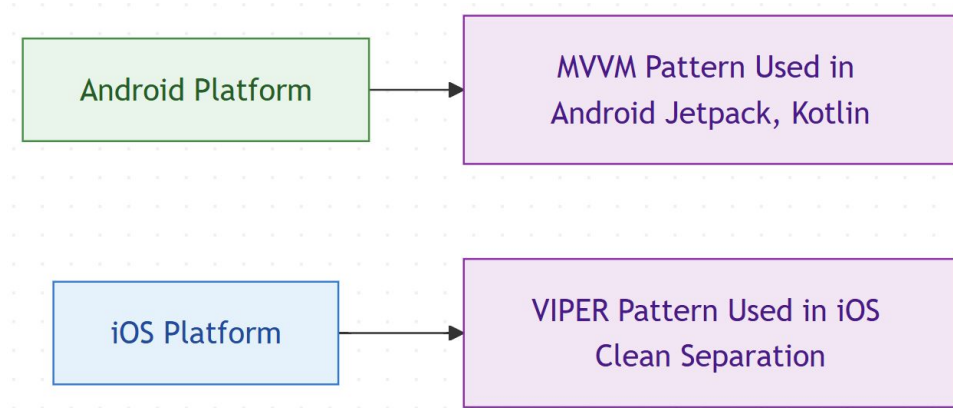
Mobile UIs – Challenges & Characteristics

- Resource-constrained: memory, battery, CPU
- Must handle lifecycle events (pause, resume, rotate)
- Navigation is central to UX
- Native APIs and platform SDKs matter
- Testing and debugging are harder



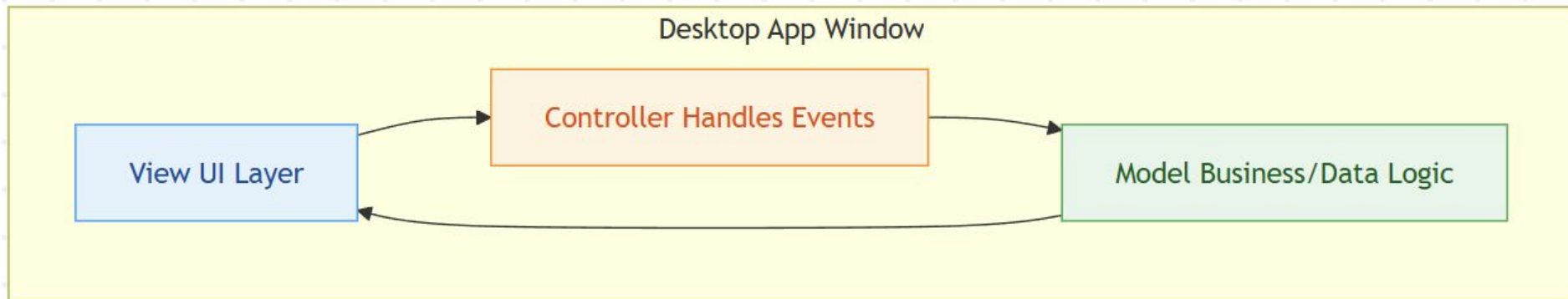
Recommended Mobile UI Patterns

- MVVM: Popular in Android (Jetpack, Kotlin)
- VIPER: Common in iOS (clear separation, testability)
- MVP: Legacy Android projects still use it
- Choose patterns that isolate UI logic and simplify testing



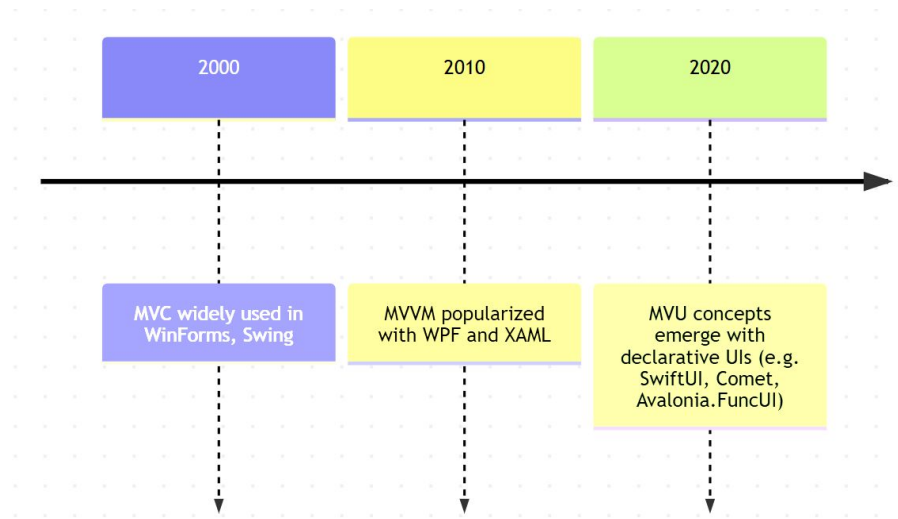
Desktop UIs – Classic Yet Complex

- Long-running sessions with complex state
- Heavy use of keyboard/mouse & multitasking
- Often use rich components (grids, trees, menus)
- Legacy systems may use MVC heavily
- Less frequent deployment → demand for stability



Recommended Desktop UI Patterns

- MVC: Still dominant in .NET, Swing, Qt
- MVVM: Preferred in WPF, modern .NET
- Declarative UIs (like SwiftUI) gaining traction even on desktop
- Separation of concerns improves debugging & extension



Decision Factors for Choosing a Pattern

- Platform capabilities & constraints
- Team familiarity and skillsets
- Testing and debugging needs
- Tooling and framework support
- Long-term maintainability

MVC

- ✓ Separation
- ✓ Testable

⚠ Moderate Complexity

MVP

- ✓ Good Separation
- ✓ Testable

⚠ Medium Complexity

MVVM

- ✓ ✓ Strong Separation
- ✓ ✓ Testable

⚠ ⚠ Higher Complexity

MVU

- ✓ ✓ ✓ Reactive Separation
- ✓ ✓ Testable

⚠ ⚠ High Complexity

VIPER

- ✓ ✓ ✓ Very Strong Separation
- ✓ ✓ ✓ Highly Testable

⚠ ⚠ ⚠ Very Complex

Summary - Selecting UI Patterns by Platform

- Platform influences UI architecture choice
- MVU/MVVM shine on Web; MVVM/VIPER on Mobile
- MVC/MVVM still strong on Desktop
- Balance structure with practicality
- Adapt patterns, don't blindly apply them
- What's next:
 - Architectural Decision-Making & Trade-Offs